

ALBERT: An Automatic Learning Based Execution and Resource Management System for Optimizing Hadoop Workload in Clouds

Chen-Chun Chen¹, Kai-Siang Wang², Yu-Tung Hsiao³, Jerry Chou^{4,*}

No. 101, Section 2, Kuang-Fu Road, Hsinchu, Taiwan 30013, R.O.C.

Abstract

Hadoop is a popular computing framework designed to deliver timely and cost-effective data processing on a large cluster of commodity machines. It relieves the burden of the programmers dealing with distributed programming, and an ecosystem of Big Data solutions has developed around it. However, Hadoops job execution time can greatly depend on its runtime configurations and resource selections. Given the more than 100 job configuration settings provided by Hadoop, and diverse resource instance options in a cloud or virtualized computing environment, running Hadoop jobs still requires a substantial amount of expertise and experience. To address this challenge, we apply a deep neural network to predict Hadoops job time based on historical execution data, and propose optimization methods to reduce job execution time and cost. The results show that our prediction method achieves almost 90% time prediction accuracy and clearly outperforms three other state-of-the-art regression-based prediction methods. Based on the time prediction, our proposed configuration search method and job scheduling algorithm successfully shorten the execution time of a single Hadoop job by more than a factor of 2 and reduce the time of processing a batch of Hadoop jobs by 40% ~ 65%.

*Corresponding author

¹ccchen@lsalab.cs.nthu.edu.tw

²kswang@lsalab.cs.nthu.edu.tw

³yu-tung@lsalab.cs.nthu.edu.tw

⁴jchou@lsalab.cs.nthu.edu

Keywords: Data analytic, Deep learning, Time Prediction,
Optimization, Job Scheduling

1. Introduction

Nowadays, Big Data has emerged as a key ingredient for success in many businesses and research domains. According to the latest report from IDC (International Data Corporation) [1], the global market for Big Data, estimated
5 at 70.5 billion in the year 2020, is projected to reach a revised value of 243.4 billion by 2027, growing at a CAGR of 19.4% over the analysis period spanning 2020-2027. The growing need to handle high volumes of data, and unstructured data, has resulted in the development of a broad set of software tools for data processing and analysis. One of the most popular solutions is Apache Hadoop [2]
10 an open-source implementation of MapReduce [3] which is designed to deliver timely and cost-effective data processing capability using commodity machines. Many software components have been built around the core services of Hadoop and formed an ecosystem to address various Big Data-related challenges. These complementary tools include HBase [4] for unstructured or semi-structured data
15 storage, Giraph [5] for graph processing, Hive [6] for query analytics, and Mahout [7] for machine learning, etc. Hence, Hadoop has been widely seen as the de facto standard for Big Data processing and is quickly becoming the main workload handler in many data centers and cluster computing systems.

While it has become rather convenient to generate a Hadoop job by using the simple but powerful MapReduce functional programming interface or
20 through the higher-level language libraries such as Hive and Mahout, it remains challenging to manage and optimize the execution of Hadoop jobs. As shown in the previous literature [8, 9, 10, 11], Hadoop has hundreds and hundreds of runtime configuration parameters to control job executions, such as the number of reducers, shuffle buffer size, and sorting percentage, to name but a few.
25 The setting of these configurations can have significant impacts on the execution time. For instance, previous study [10] reported that a factor of 2 to 8

speedup could be obtained under the same amount of resource allocation by simply choosing the proper configuration settings. However, in reality, Hadoop
 30 jobs are often written or generated by users from different application domains with little or no knowledge of the framework. Thus, jobs are often run with the default configuration without being properly optimized.

The job execution problem is further complicated by the fact that modern data centers often follow the cloud paradigm and deploy a virtualization layer
 35 using virtual machines or containers to improve the flexibility and utilization of computing resources. While virtualization technologies allow users to freely specify the resource requirements of their compute instance, this also becomes another job execution configuration that needs to be carefully chosen, especially when jobs need to be run in a cost-effective manner on a pay-as-you-use public
 40 cloud platform or in a resource-limited in-house cluster. Last but not least, in order to maximize the overall system throughput and resource utilization, we must rely on an intelligent system-level job scheduling strategy to decide the job execution order properly. An application which is oblivious to the scheduling algorithm, such as the naive First In First Out (FIFO) strategy, may miss the
 45 opportunity to fully optimize system performance.

To overcome the aforementioned problems in Hadoop execution optimization, we present *ALBERT* (i.e., named after Albert Einstein) an **A**utomatic **L**earning-**B**ased **E**xecution and **R**esource Managemen**T** System. *ALBERT* consists of two main components: a *predictor* that accurately estimates the job ex-
 50 ecution with a given job profiling, Hadoop configuration, and resource requirement; and an *optimizer* that utilizes the job time prediction to automatically optimize the execution performance both at the individual job level as well as at the overall system level. Different from the traditional prediction methods, which rely on simulations [12] or linear regression models [13], we use the deep
 55 neural network (DNN) technique to predict Hadoops job execution time, while we also built our whole management system around this data-driven learning-based approach. Although the DNN model has been actively studied in several application domains [14, 15, 16], only a few attempts have been made [17, 18]

for the purpose of job execution time prediction. However, we expect that this
60 approach could be even more successful in predicting Hadoops job execution
time because data analytic applications often repeatedly run the same job with
different data inputs or resource configurations. Thus, historical job execution
logs can be easily collected for training job time prediction models and driving
job execution optimization.

65 To illustrate the performance benefits of our proposed approach, we built
our solution on an OpenStack [19] private cloud platform to manage its Hadoop
job execution workload. The job-level optimizer is a 2-step approximate search
algorithm that can determine the optimal Hadoop runtime configurations for
either minimizing the job execution time under a given cost budget or minimiz-
70 ing the job execution cost (i.e., resource usage) under a given job deadline. On
the other hand, the system-level optimizer is a job scheduler that aims to max-
imize resource utilization and system throughput by formulating the problem
as a 2D bin-packing problem. Using the job execution dataset collected from
the real system, we showed that our DNN model can achieve 89% prediction
75 accuracy on job execution time, while previous prediction methods exhibited a
much worse performance. Based on the accurate job time prediction, the job-
level optimizer can achieve 2x performance speedup for individual job execution,
and reduce the total execution time of running a mixed Hadoop job workload
by 40% $\sim$ 65%. More importantly, all of these optimization results were pro-
80 cessed automatically by our proposed system, *ALBERT*, using a historical job
execution dataset without prior job information or human intervention.

The remainder of this paper is structured as follows. Related work is dis-
cussed in Section 2, following which Section 3 describes the design and im-
plementation of our proposed system, *ALBERT*. Sections 4 and 5 detail our
85 prediction and optimization methods, before the experimental setup and results
are presented in Sections 6 and 7, respectively. Finally, the paper is concluded
in Section 8.

2. Related Work

A recent survey [20] has summarized all of the above-mentioned techniques
90 for Big Data processing, including batch data processing (e.g., Hadoop MapRe-
duce, Spark) and streaming data processing (e.g., Apache Storm, Flink, Samza).
Depending on the job execution performance which is predicted, the survey [20]
classified these techniques in several categories: rule-based [21], cost model-
ing [22], simulation-based [23, 12], experiment-driven [13, 24, 25, 11], machine
95 learning [10, 26, 8], and adaptive [27]. The first three categories (rule-based,
cost modeling, simulation-based) can be considered as white-box approaches,
which require more explicit information and understanding of the computing
frameworks, such as Hadoop, to build the prediction model. In contrast, the
latter two categories (experiment-driven, machine learning) can be considered
100 as gray-box or black-box approaches, which build prediction models based on
jobs profiling results or execution logs without knowing how the computing
framework works. Finally, the adaptive approaches focus on the techniques for
performance monitoring and reconfiguration control which could be applied to
all other prediction-based techniques.

105 In general, all the methods besides machine learning use prior knowledge
or observation of the execution system to construct an equation to present the
performance model, and employ the linear regression algorithm to calibrate
the coefficients and parameters of the equation. On the other hand, machine
learning approaches tend to use more sophisticated learning methods to train
110 the performance prediction model without pre-defining the modeling equations.
Most of the existing machine learning approaches are based on three types of
learning algorithms: (1) decision-tree [10] or random forest [26, 28]; (2) SVM
(Support Vector Machine) [8] or SMR (Support Vector Regression) [29]; and
(3) Multilayer Perceptron (MLP) [9]. Our experiment results prove that deep
115 neural networks can achieve more accurate performance predictions and thus
achieve better performance optimization results.

Some of these works [12, 9, 11] aim to use the performance predictor to

analyze the impacts from various execution factors, such as resource allocation, configuration setting, data placement, and node failures. Others aim to use the
120 predicted performance to find the proper resource allocation (i.e., number and type of VMs) and Hadoop configuration parameters for executing a MapReduce job to achieve a service-level objective, such as minimum execution time [10, 13, 26, 27, 29], cost [8], resource consumption [24], and service-level-agreement [22, 23]. However, all of these focus on the optimization of single job execution.
125 In this work, we further optimize the overall system throughput and resource utilization by constructing the scheduling problem as a 2D variable size bin-packing problem according to the predicted job execution time.

Deep learning has been applied for performance prediction in various application domains, including parallel applications in computing clusters [30],
130 workflow applications on the cloud [31], cloud services in data centers [32], and thread programs on heterogeneous CPUs [33], etc. While the neural network models used in these works are similar, the deep learning technique is applied to address different application-specific challenges. In our work, we use the neural network to predict job performance under various job configurations. In addition,
135 we present how the predicted job information can be used to optimize the individual job and overall system performance.

3. Design and Implementation

3.1. On-demand Hadoop Execution Environment

There could be many different ways to set up the system environments for
140 running Hadoop jobs. In this work, we consider the case of running on-demand Hadoop jobs on a virtualized resource pool as shown in Figure 1. This system environment is commonly seen in cloud platforms to provide cost-effective Big Data analytic services, such as the EMR (Elastic MapReduce) [34] service from AWS (Amazon Web Services) [35], or the Sahara project [36] in OpenStack [19].

145 There are many different ways to set up the system environments for running Hadoop jobs. In this work, we consider the case of running on-demand

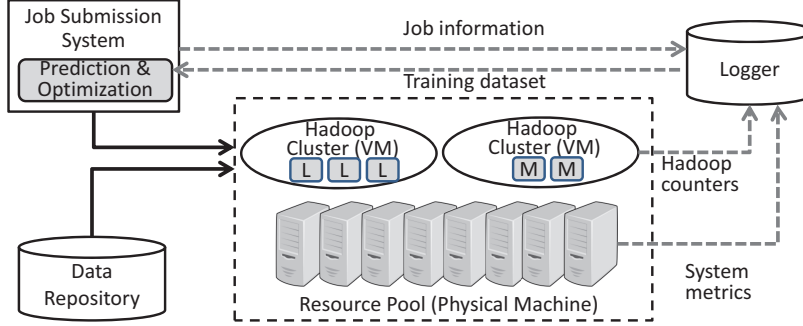


Figure 1: System overview. Hadoop clusters are launched on-demand to process user jobs. Hadoop configuration and VM (Virtual Machine) instance type can be tuned to optimize job running performance or cost. Four types of data are collected to train our prediction model, naming the job information, data information, Hadoop counters, and system metrics.

Hadoop jobs on a virtualized resource pool, as shown in Figure 1. This system environment is commonly seen on cloud platforms, with its purpose being to provide cost-effective Big Data analytic services, such as the EMR (Elastic MapReduce) [34] service from AWS (Amazon Web Services) [35], or the Sahara project [36] in OpenStack [19].

In this system environment, users submit their data analytic workload as a set of Hadoop jobs. Each job is executed in an on-demand virtual cluster environment that has the same lifetime as the job, meaning the compute instances of the cluster are launched when the job execution starts, and terminated when the job execution ends. Users control their job resource usage by choosing the number of compute instances and the type of compute instance in the cluster. This decision is referred to, in the remainder of the current paper, as the **resource configuration**. For instance, on the AWS cloud platform, different instance types (Large, Medium, and Small) are offered according to the size of resources on a virtual machine (VM) in terms of the number of CPUs, memory size, and network bandwidth, etc. Hence, using more or larger compute instances can obtain higher computing power, but also be more expensive and resource-consuming.

Besides resource configuration, Hadoop has hundreds and hundreds of run-

time configuration parameters that can be adjusted by users for job execution optimization. According to a previous study [37], approximately 30 of these parameters can have a substantial effect on job performance. Hence, the decision for these configuration parameters is referred to as the **Hadoop configuration** in the rest of the paper. The parameters considered in this work, alongside their value ranges, are listed and summarized in Table 1.

The goal of our work is to optimize the Hadoop execution from the perspectives of both users and system managers (or cloud service providers). Indeed, users want to find the optimal resource spec and runtime configuration settings for optimizing the execution time or cost of individual jobs. In contrast, system managers are more interested in improving the job scheduling decision to deliver higher system throughput and resource utilization. Therefore, we developed *ALBERT*, an automatic learning-based execution and resource management system for Hadoop. As shown in Figure 1, this approach is driven by a learning-based job prediction method powered by the historical job execution data, so that job execution time under different *Hadoop configuration* and *resource configuration* settings can be obtained in advance in order to find the best job setting and scheduling decisions. We detail our data collection and system implementation in the following sub-sections before introducing the prediction and optimization methods in Section 4 and Section 5.

3.2. Data Driven and Self-Learning Approach

ALBERT is a system optimization approach driven by a self-learning time prediction method using historical job execution data. We believe this approach is particularly suitable for managing Hadoop execution jobs, because most users employ similar tools or programs to generate Hadoop jobs and analyze data. For instance, Mahout [7] is one of the famous libraries built on top of Hadoop to support scalable machine learning applications. There are only a handful of machine learning algorithms that are implemented in Mahout, such as K-means and

Table 1: Hadoop configuration and the range of its parameter values considered in the paper.

configuration	default	min	max
job.reduces	1	1	128
tasktracker.map.tasks.maximum	2	2	12
tasktracker.reduce.tasks.maximum	2	2	12
task.io.sort.mb	100	100	150
task.io.sort.factor	10	10	100
reduce.shuffle.parallelcopies	5	5	20
reduce.shuffle.input.buffer.percent	0.7	0.5	1
reduce.shuffle.merge.percent	0.66	0.5	1
reduce.shuffle.memory.limit.percent	0.25	0.2	0.5
map.sort.spill.percent	0.8	0.5	0.9
reduce.input.buffer.percent	0	0	1
reduce.merge.inmem.threshold	1000	500	2000
job.reduce.slowstart.completedmaps	0.05	0	1
jobtracker.handler.count	10	10	100
tasktracker.http.threads	40	40	60
map.cpu.vcores	1	1	4
reduce.cpu.vcores	1	1	4
map.memory.mb	1024	1024	2048
reduce.memory.mb	1024	1024	2048
app.mapreduce.am.resource.mb	512	1024	2048
app.mapreduce.am.resource.cpu-vcores	1	1	4
child.java.opts	-Xmx200m	-Xmx200m	-Xmx1024m

195 stochastic SVD. Hence, users essentially keep running the same set of programs
on different data inputs. In the data warehousing system, OLAP (On-Line Analytical Processing) applications also periodically perform the same operations over daily updated datasets to refine their output decision. Therefore, there is a greater likelihood of collecting a sufficient amount of job execution history of
200 a Hadoop program for job time prediction.

As shown in Figure 1, our system collects four types of data with respect to single job execution, namely *job descriptions*, *Hadoop counters*, *system metrics*, and *data characteristics*. These data types are combined into a job profile to drive our time prediction and optimization methods. We summarize all the
205 attributes in a job profile in Table 2 and briefly describe them as follows: 1) **Job descriptions** refer to the data collected from the job submission system. The data contains the overall job execution information, such as job status, and job submission/start/finish time, as well as certain program or user information, such as executable filename and userID, etc.; 2) **Hadoop counters** are
210 the statistical numbers collected by the Hadoop framework during job execution. Tens and tens of built-in counters are provided by Hadoop, and custom counters could also be added. This set of data is critical for characterizing the runtime behavior of a job, especially in terms of the data size generated by execution phases; 3) **System metrics** contain the resource usage information
215 collected from the monitoring system, such as CPU utilization, memory usage, disk read/write size and network I/O bandwidth, etc. Each metric contains a set of time-series measurements from compute nodes. To limit the size of our job profile, our prediction analysis only uses the aggregate statistic numbers (i.e., max/min/mean) over 4 time periods for each system metric; 4) **Data characteristics**
220 **are** obtained from the analysis of a small subset of randomly sampled data files, and include, for example, the number of lines, the number of words per line, etc. This information allows us to have a more detailed characterization of input data besides its file size.

Table 2: Data Collections of Job Profile

Category	configuration
Job Descriptions	job name
	VM type
	VM number
	job status (succeeded/failed)
Hadoop Counters	mapreduce.job.reduces
	mapreduce.task.io.sort.mb
	mapreduce.task.io.sort.factor
	...
System Metrics	CPU usage {sum,avg,max,min,stddev}
	Memory usage {sum,avg,max,min,stddev}
	Disk {read, write} {sum,avg,max,min}
	Network {input, output} {sum,avg,max,min}
Data Characteristics	input data size
	number of input data file
	#word/KB
	#line/KB
	#word/line in {avg,stddev}
	#char/line in {avg,stddev}
	#char/word in {avg,stddev}

225 3.3. Implementation on Cloud Platform

To validate our approach, we built *ALBERT* on an open-source cloud platform software program, OpenStack [19], which has been widely used to build private and public cloud platforms [38]. OpenStack has a modular architecture to allow the open-source community to develop its components in separate projects. The core components provide IaaS (Infrastructure-as-a-Service) service to create virtual machines. For our work, we also installed several other components and combined them to realize the system shown in Figure 1. These components include *Sahara*, a Hadoop cluster deployment service; *Heat*, a resource orchestration program; and *Ceilometer*, a resource monitoring system.

235 *Sahara* plays the role of a submission system that launches Hadoops jobs and its virtual clusters using the orchestration template managed by *Heat*. In the original software, users must specify the Hadoop configuration, VM instance type, and number when submitting their jobs to *Sahara*. However, in our im-

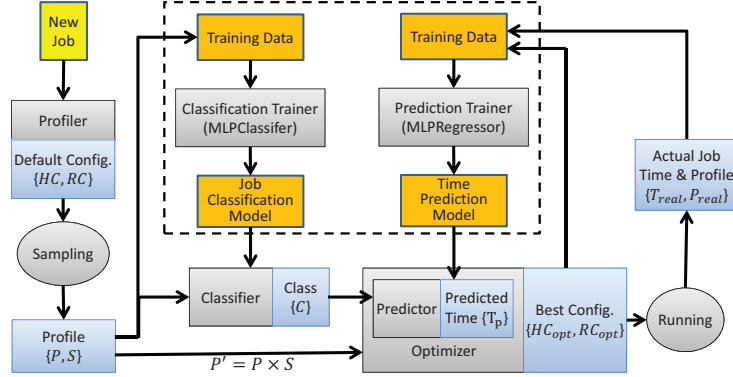


Figure 2: Our learning-based prediction. HC is a Hadoop configuration, RC is a resource configuration, P is a job profile, α is a scaling vector, C is an application class, and T is job execution time.

plementation, we added a prediction and optimization plug-in component to
 240 determine proper configurations automatically without human intervention. To
 collect the profile of any running jobs in the system, we implemented a daemon
 running on the Hadoop master node to collect job descriptions from *Sahara*, sys-
 tem metrics from *Ceilometer*, Hadoop counters from Hadoop logs, and finally
 the data characteristics from analysis of several 1KB sample data chunks of job
 245 input files. This job profile data was then stored in a database and periodically
 pulled out to build the prediction models and drive optimization methods, as
 described in the following sections.

4. Deep Learning Job Time Prediction Method

The details of our data-driven learning-based prediction method are depicted
 250 in Figure 2. It consists of an *offline* model training process and an *online* model
 prediction process. The offline model training process is indicated by the dotted
 box in the figure, and its training dataset is collected from the historical job
 executions, as detailed in Section 3.2. The online model prediction process
 comprises three steps: job profiling, job classification, and job prediction. Each
 255 of these steps is handled by a component introduced as follows.

4.1. Profiler

Upon receiving a job from users, the Profiler takes the responsibility of generating its job profile. The profile is used by the Classifier and Predictor for online job classification and execution time prediction. The job profiles are also
260 stored and periodically merged into the training datasets so as to build future classification and prediction models. As detailed in Section 3.2, a job profile consists of four types of collected data, most of which can only be collected during job execution time. Therefore, the main challenge for the Profiler is to obtain this runtime job execution information without introducing too much
265 profiling overhead or delay before job execution.

Our approach is to collect job profiling information by running small sample jobs which have a downsized input dataset randomly selected from the original job input file. The sample jobs are run with a default Hadoop and resource configuration setting, $\{HC_{def}, RC_{def}\}$, and the sampling time can be made as
270 short as it needs to be by choosing the proper data sampling ratio. To capture the influence of data downsizing, we profile a job with two different sampling data sizes to determine a scaling vector, S , so that the profiling values of the original data size can be estimated as $P = P' \times S$, where P' is a vector of profiling values from the sample job, and P is a vector of the original user job.

275 4.2. Classifier

A variety of applications can run on Hadoop. Although they all have to follow the MapReduce computation model, their execution behaviors can sometimes still vary widely. Therefore, we decided to categorize jobs into several classes, and then build a time prediction model for each class individually. As
280 shown by our evaluation in Section 7.1, classification can effectively improve prediction accuracy. Hence, we need to have a classifier that can identify the class of a newly-arriving job according to its job profile.

The classifier is built offline by the training dataset produced from running a set of sample jobs. Each sample job is a record in the training dataset. The
285 profiling information of a job is its feature, and the application type of the

job is its label. Thus, the model can help us to identify the type of each job based on its profiling information. We apply a fully connected neural network model by using *Scikit-Learn MLPClassifier*(*multi-layer perceptron classifier*) in our implementation. After our tuning, we set the number of hidden layers to 3,
290 and the number of neural in each layer to 100. We adopt *adam* as the optimizer, and the activation function is *ReLU*. We also set the batch size to 125, alpha to 0.003, and the initial value of the learning rate to 0.002. Other settings are default values.

4.3. Predictor

295 The role of the Predictor is to predict the execution time of a job. It can be formally described as a function below:

$$predictor(P, HC_i, RC_i) = T_p \quad (1)$$

where P is the profile of a job, and HC_i , RC_i are the Hadoop configuration and resource configuration to run the job. The predicted time T_p is then used by our optimization strategies described in the next section to drive the search
300 for the optimal Hadoop and resource configuration HC_{opt} , RC_{opt} for single job execution.

Since the sample jobs only run with the default configurations, the training dataset for the prediction model is collected from the actual job execution instead, that includes $\{HC_{opt}, RC_{opt}, P_{real}, T_{real}\}$ as shown in Figure 2. Similar
305 to the classification model, we use a fully connected neural network to train the prediction model. The difference between the Classifier and the Predictor doesn't have an activation function in the output layer, and so the output value is a continuous value that can be used for regression. After our tuning, we set the number of hidden layers to 4. The number of neurons in each layer is only
310 half that of the previous layer, which makes the shape of the network a pyramid. We adopt *adam* as the optimizer, and the activation function is *ReLU* except for the output layer. To prevent overfitting, we adopt regularization and set the l2 norm value to 0.001. Other settings are default values.

5. Job and System Optimization Methods

315 Taking advantage of job time prediction information, we further propose job execution optimization techniques performed at both job and system levels as follows.

5.1. Job Level Optimization

The goal of our job level optimizer is to determine the best Hadoop and resource configuration $\{HC_{opt}, RC_{opt}\}$ for running a job with respect to execution 320 time and cost. However, in an on-demand resource system environment, such as the cloud, minimizing time and cost together presents a trade-off problem. That is, running jobs with more resources can reduce execution time, but also requires higher execution costs. On the other hand, running jobs with fewer 325 resources saves cost, but causes a longer execution time. Therefore, our job optimizer is designed to be capable of arbitrary taking one (either time or cost) as the minimizing objective, and the other one as the constraint when determining the best job execution configuration.

Intuitively, our optimization method is based on search algorithms that 330 compare the execution time and cost among all the configuration candidate solutions. Consider any given setting of Hadoop and resource configuration $\{HC_i, RC_i\}$. Its corresponding execution cost can be directly determined by the number of compute instance (i.e., $RC_i.instance_num$) and the cost of the compute instance type (i.e., $RC_i.instance_cost$) chosen from the resource configuration. On the other hand, its corresponding execution time can be estimated 335 by our job predictor as shown in Eq. 1. Therefore, in the search process, we can filter the candidates according to the time or cost constraint, and keep the solution with respect to the minimizing objective.

The challenge of our formulated search problem is that the search space 340 could be vast because of the number of Hadoop configuration parameters, and types of compute instance which need to be considered in the optimization. Furthermore, the resource configuration and Hadoop configuration settings cannot

Algorithm 1: 2-step approx. search job optimization

```

1 Function job_optimizer(P, Cons)
    Data: P is a job profile, Cons is the constraint of execution time (or cost)
    Result: The optimal Hadoop and resource configuration  $\{HC_{opt}, RC_{opt}\}$  with
        the minimum execution cost (or time)

    /* Global Search */
2    Randomly sample a set of configurations:  $CF = \{CF_i | CF_i = \{HC_i, RC_i\}\}$ ;
3    Compute the execution time and cost of each  $CF_i$ :  $t_i = predictor(P, HC_i, RC_i)$ 
         $c_i = RC_i.instance\_num \times RC_i.instance\_cost$ ;
4    Find the best  $CF_{opt} \in CF$  where  $t_{opt}$  is the minimum, and  $c_{opt} \leq Cons$ ;

    /* Local Search */
5    Let  $FC_{opt}$  be the initial search point;
6    Replace  $FC_{opt}$  with a better optimal setting using the gradient boosted trees
        algorithm;
7    All configurations found in the search process must satisfy the search constraint
        Cons;
8    return  $\{HC_{opt}, RC_{opt}\}$ ;

```

be searched independently because they do have performance influences on each other. As a result, brute force search is not feasible. Therefore, in this work, we

345 use a *2-step approximate search algorithm* which consists of a *global searching* step and a *local searching* step as shown in Algorithm. 1. In the global searching step, a search of the entire global solution space is performed by randomly generating a set of configurations. The parameter values in these configurations are uniformly sampled from their corresponding valid value ranges. Subsequently,

350 all of these sampled configurations are evaluated by the optimization objective function, and that with the best result is selected as the global best. In the local searching step, the global best found from the first step is inputted as the initial search position. Following this, the gradient boosted trees algorithm is used to search the local minimum with respect to the global best position. In the

355 implementation, we use the python optimization package, *Scikit-Optimize*, and call the functions of *dummy_minimize* and *gbrt_minimize* for global searching and local searching, respectively.

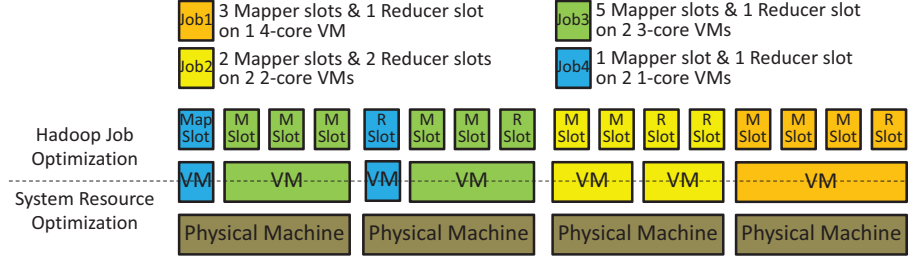


Figure 3: Resource usage mapping in an on-demand Hadoop execution environment from Hadoop jobs to physical resource pool.

5.2. System Level Optimization

While the job level optimization can minimize the execution time and cost of individual jobs, it does not necessarily guarantee the overall performance of the whole system. Therefore, we further develop a scheduling algorithm to minimize the total execution time (i.e., makespan) of running a set of jobs.

As depicted by the resource mapping of an on-demand Hadoop execution environment in Figure 3, jobs are containerized and run inside their own on-demand Hadoop cluster instance, so they can be executed with the different numbers of mapper (or reducer) slots and types of virtual machine instances according to their own job level settings of Hadoop and resource configurations. The mapping between map (or reducer) slots and the virtual machine is managed by the Hadoop resource manager, like YARN [39]. However, the mapping between virtual machines and physical machines is controlled by our scheduler.

We formally define our system-level optimization problem as follows. Given a set of jobs that are allowed to be run within a range of resource and Hadoop configuration settings specified by users, our goal is to minimize the time needed for completing all these jobs by deciding the Hadoop and resource configurations of each job, and finding the placement of each corresponding VMs on a set of limited physical machines.

We solve our system-level optimization problem by formulating it into a variant of the bin packing problem. As shown by the example in Figure 4, we consider each CPU core as a bin and each job’s VM as an object that needs to

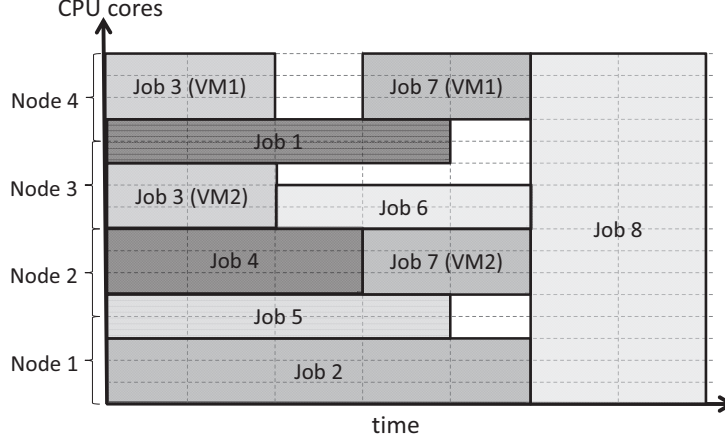


Figure 4: Resource allocation optimization formulated as a 2D variable sized bin packing problem.

be placed into the bins. Each object has the shape of a 2D rectangle where the width is the job execution time and the height is the number of CPU cores per VM. Therefore, we refer to our problem as a *2D variable-sized bin packing problem (2DVSBPP)*, where the name '*variable-sized*' refers to the shape of rectangles, which can be adjusted in our optimization problem by choosing different job level configuration settings.

There are previous literature [40] discussing the solutions of the variants of the 2D bin packing problem. D. Pisinger et. al. [41] presented an integer-linear formulation for the problem, and provided a lower-bound using the Dantzig-Wolfe decomposition. An exact algorithm was proposed, but the time complexity was rather expensive. Therefore, we aim to propose a greedy algorithm, which is based on the 2D rotatable rectangle bin packing problem discussed in the previous work [42], where the rectangles can be rotated and are either placed upright or sideways into bins. Our problem is more complicated because the rectangle shape can be changed in multiple ways depending on the job configurations, and the placement of rectangles must be subject to the job scheduling constraint. That is, the VMs from the same job must have the same starting time, and the CPU cores allocated to a VM must be from the same physical

machine.

Our algorithm follows the placement principles below: (1) Give higher schedul-
400 ing priority to the larger jobs where the size of jobs is determined by the pre-
dicted job execution time under default resource and Hadoop configurations; (2)
Minimize the execution time of the larger jobs to reduce the total job execution
time, and schedule them on VMs with the greedy algorithm mentioned above;
(3) Adjust the resource configuration of smaller jobs to fill the space and utilize
405 the residual capacity; (4) Control the ratio between large and small jobs by a
parameter p .

When the value of p is too large or too small, this will lead to sub-optimal
system performance. For instance, a larger p indicates most jobs will be con-
sidered as the large jobs in the first phase of our scheduling algorithm. That
410 means that more jobs will be optimized with no restriction, and so each job
tends to use more resources. This may lead to the individual job running faster
with more resources, but harm the overall throughput. Moreover, fewer jobs
are left for filling the space so as to maximize resource utilization. On the other
hand, if p is too small, most jobs cannot be run with their best configuration,
415 and so the individual job time can be prolonged. As a result, although we have
more jobs which can be used to fill the space, the overall system performance
cannot be optimized. Overall, we observe that the resulting total job execu-
tion time is a convex function of p . Therefore, a binary search strategy can be
applied to determine the best setting of p in our scheduling approach. Based
420 on the aforementioned principles, we propose a system-level optimization algo-
rithm described in Algorithm 2. The input J is a set of jobs to be scheduled;
 B is a class of bins to maintain the scheduling status, including occupied core
slots and free space; p is the user-defined ratio that determines how many small
jobs there are for space-filling. Our proposed scheduling algorithm consists of
425 the following five steps.

Step1. Calculate job size: We determine the job size based on predicted
job execution time under the default resource and Hadoop configurations (line
4 in Algorithm 2).

Algorithm 2: System-level optimization

```

1   $J$ : the jobs to be scheduled;
2   $B$ : a class of bins to maintain the scheduling status, including occupied core slots and
   free space;
3   $p$ : the ratio of small jobs for space filling;
   /* Step1. Calculate job size */
4  Sort the jobs in  $J$  based on the predicted execution time under default  $RC$  and  $HC$  in
   descending order;
   /* Step2. Divide job groups */
5   $x = |J| \times (1 - p)$ ;
6   $G_L = \{j_1, \dots, j_x\}$ ;
7   $G_S = \{j_{x+1}, \dots, j_{|J|}\}$ ;
   /* Step3. Optimize the large jobs */
8  for  $j \in G_L$  do
9       $Cons = null$ ;
10      $\{HC_{opt}, RC_{opt}\} = job\_optimizer(j, Cons)$ ;
11      $j.width = RC_{opt}$ ;
12      $j.length = predictedjobtimefromjob\_optimizer$ ;
13 end
   /* Step4. Pack the large jobs first */
14  $bin\_packing(G_L, B)$ ;
   /* Step5. Fill space by small jobs */
15 Let  $S$  be the set of free space from  $B$ ,  $S$  are sorted by the start time of each free space;
16 for  $j \in G_S$  do
17     for each space  $s \in S$  do
18          $Cons = (s.width, s.length)$ ;
19          $\{HC_{opt}, RC_{opt}\} = job\_optimizer(j, Cons)$ ;
20         if  $j$  can be place in  $s$  then
21              $bin\_packing(j, B)$ ;
22             Update  $S$ ;
23             break;
24         end
25     end
26 end

```

Step2. Divide the jobs into two groups by their sizes: The first
430 group, G_L , contains the larger jobs whose sizes are ranked in the top $(1 - p\%)$

(line 6 in Algorithm 2); the second group, G_S , contains the rest of the smaller jobs whose sizes are ranked in the bottom $p\%$ (line 7 in Algorithm 2).

Step3. Optimize the configurations for the jobs in the first group (G_L):

For the larger jobs, we want to minimize their job execution time. Thus, we use
 435 Algorithm 1 to find their optimal Hadoop and resource configurations that can minimize their execution time without cost constraints (line 10 in Algorithm 2).

Step4. Schedule the jobs in the first group (G_L): We use a greedy 2D bin packing algorithm to place the jobs from the first group onto compute nodes, so that the latest job completion time can be minimized (line 14 in Algorithm 2).

440 **Step5: Fill the space by the jobs in the second group (G_S):** Let S be the set of free space from the bins after placing the first group of jobs. We try to fill the space from the largest hole to the smallest hole using the jobs in the second group. For each free space $s \in S$, we use its width (available resources) and length (available time) as the constraints to find a job (using Algorithm 1)
 445 that can be fitted into the hole (line 18 and 19 in Algorithm 2). If the job can be optimized and fit the space, we fill the hole and update the set S (remove the used space and update the remaining space according to the start time of the space) (line 20 to 24 in Algorithm 2), and then move on to the next job. Finally, all the jobs in G_S are going to be fitted into the holes; if there is no
 450 space, the jobs will append to the tail of all the jobs (consider the space after the end of all jobs as a big hole).

In the proposed system-level optimization algorithm, we rely on the 2-step approximate search job optimization (Algorithm 1) and the 2D variable-sized bin packing algorithm as the underlying algorithm. For Algorithm 1, users can
 455 decide the search granularity and change the underlying algorithm or implementation for global and local search to get better results. On the other hand, the 2D variable-sized bin packing problem is a NP-complete problem. Therefore, we analyze our proposed strategy shown in Algorithm 2 as follows. The optimization of jobs in G_L requires, at most, n iterations for Algorithm 1, where n is the
 460 total number of jobs. Moreover, the bin packing algorithm is also associated with n . For optimizing jobs in G_S , for each job, it takes at most $|S|$ iterations

Table 3: Types of VM instances.

Type	CPU Cores	Memory Size	Disk Size	Normalized Cost
Small	1	2GB	100GB	1
Medium	3	6GB	300GB	3
Large	6	12GB	600GB	6

to conduct Algorithm 1. The forming of a free space requires at least 2 jobs creating a hole, and so the number of $|S|$ is not going to exceed n . The bin packing and the update steps in the if-statement only take linear time, and so eventually, it will be no need for more than n iterations. The sorting processes is $n \log(n)$. Hence, considering the whole algorithm, the complexity is $O(n^2)$ times the complexity of Algorithm 1 plus the complexity of the 2D variable-sized bin packing algorithm, $O(n)$.

6. Experimental Setup

6.1. Environment & Setup

We conducted experiments on an OpenStack cloud platform built by an in-house cluster with 8 physical nodes. Each node has 2 Intel Xeon X5670 2.93GHz CPUs and 96GB memory. Three types of VM instances, *Small*, *Medium*, and *Large*, are offered in the cloud platform, as shown in Table 3. The large size VM is the default configuration in our experiments.

The application workloads in our evaluation are generated from HiBench [43], which is a well-known workload generator for Hadoop. As summarized in Table 4, we consider 7 commonly-used applications from 4 different categories, including text and graph processing, sort, and machine learning. Note that for *PageRank*, *K-means Clustering*, and *TriangleCount*, their executions consist of two phases running different types of jobs. Hence, there are a total of 10 types of jobs in our experiments.

To evaluate the accuracy of our job classification model and the error rate of our time prediction model, we use the following metrics:

Job Classification

$$AvgAccu = \frac{1}{N} \sum_1^N \begin{cases} 0, & \text{if } Predict(n) \neq Actual(n) \\ 1, & \text{if } Predict(n) = Actual(n) \end{cases} \quad (2)$$

485 where N is the number of test cases.

Time Prediction

$$AvgErr = \frac{1}{N} \sum_1^N \frac{|Predict(n) - Actual(n)|}{Actual(n)} \quad (3)$$

where N is the number of test cases.

6.2. State-of-the-art Comparison

To show the advantage of the deep learning technique, we compare our prediction results to the following state-of-art and mainstream prediction methods proposed in the previous literature.

EQ represents experiment-driven approaches [24] that construct an equation to compute the execution time according to the running behavior of a MapReduce job and fine-tune the parameters of the equation by linear regression algorithms. In our experiment, we implement it according to the strategy proposed by [24] as follows. It models the time of three phases of MapReduce execution (map phase, data shuffle phase, and reduce phase) separately. The time of the map phase is proportional to the number of Map waves and input data size, where the number of Map waves is the number of map tasks divided by the number of mapper slots (i.e., computing cores). The time of the reduce phase is proportional to the number of Reduce waves and intermediate data size. The time of the shuffle phase is proportional to the ratio between the number of mappers and the number of reducers. In addition, because the execution time of the first Reduce wave can be overlapped with the time of the last Map wave, the time of the first Reduce wave, and the time of subsequent Reduce waves are also modeled separately in this method. Finally, a linear regression method is used to tune the coefficients in the model according to different types of workloads. Compared to other white-box approaches, this method already builds a more

sophisticated equation to model the execution time. However, it is impossible to construct equations to explicitly model the effect of every parameter in the Hadoop configuration. Therefore, this approach can only predict the time under different resource configurations, but not under different Hadoop configurations.

Tree-based represents the gray-box modeling approaches [10, 26, 8] based on the decision-tree or random forest machine learning techniques. In the experiment, we implement this according to [10] as follows. First, we classify configuration parameters into the parameter sets for map and reduce tasks using the random forest feature importance method. Subsequently, we build a 2-level prediction model by tree-based regression such as random forest regressor, the extra trees regressor, and the gradient boosting regressor as the ensemble method. The 2-level prediction model means that there is an additional prediction step on the intermediate features (such as the median task execution time) between the input features and the final time results. The idea of this 2-level modeling is similar to deep learning. However, instead of learning these intermediate features automatically from the training dataset, here it requires human knowledge to define these features properly.

SVM represents the approaches [8] using Supported Vector Machine(SVM). We implement SVM by applying a stepwise regression for feature selection and then using Supported Vector Machine(SVM) regression to build the time prediction model.

7. Experimental Evaluation

In our evaluation, we first evaluate the accuracy of our job classification model and the job time prediction model. Then we conduct two categories of experiments to demonstrate the benefits of our auto-tuned system based on job time prediction information. The first category is called *job-level* optimization, which is designed to optimize the execution time and cost of a single job, and

Table 4: Types of applications in the evaluation.

Category	Application	Goal
Text	WordCount	Word frequency counter
	InvertedIndex	Computing inverted index of a set of documents
Graph	PageRank	Rank web pages
	TriangleCount	Triangle counting in a graph
Sort	Sort	Sort data
	TeraSort	
Machine Learning	KMeans Clustering	Group objects with similar

the second one is called *system-level* optimization, which is designed to optimize the execution time and resource usage of the whole system.

7.1. Job Classification

Here we evaluate the benefit and accuracy of our job classification method. As mentioned in Section 6.1, there are 10 types of jobs in our experiments. Without classification, all jobs share the same time prediction model regardless of their job types. On the contrary, with classification, a prediction model is built for each type of job individually. Figure 5 is the comparison of prediction error rates between the two approaches. As shown by the results, the error rates without classification are much higher than the error rates with classification across all types of jobs. The differences range from a factor of 2.1 (i.e., Terasort) $\sim$ 9.7 (i.e., TriangleCount_1, 1st execution phase of TriangleCount) times higher. Therefore, the result indicates that the classification is necessary to ensure our prediction accuracy.

In this work, we use deep learning (neural network) to build the classification model and identify the type of any new arrival job in our system. To validate our choice, Figure 6 compares the accuracy of our classification method to several other popular classification algorithms, including 1) Nearest Neighbors classified

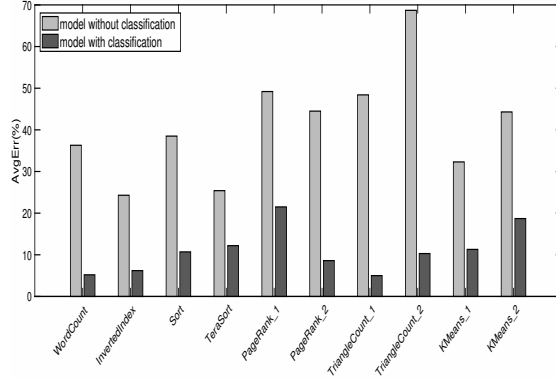


Figure 5: The comparison of error rates between the time prediction model built with job classification and without job classification.

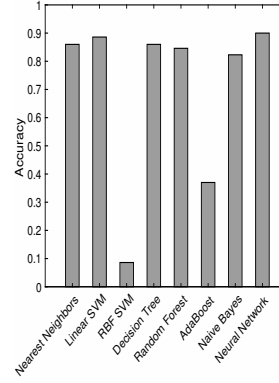


Figure 6: The accuracy comparison of classification algorithms.

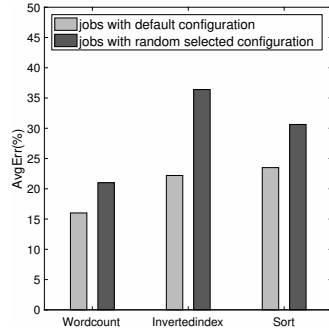


Figure 7: The impact of job configuration to the accuracy of the EQ time prediction method.

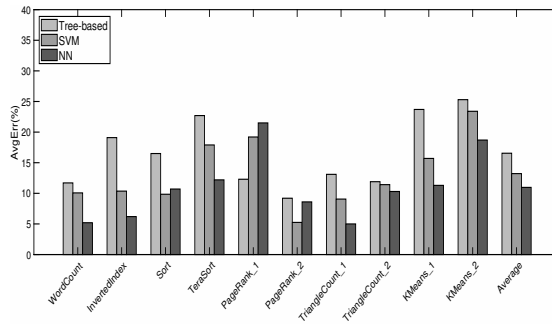


Figure 8: The comparison of prediction accuracy between our neural network method (NN) and other two machine learning methods (Tree-based, SVM).

by a majority vote of its neighbors; 2) Support Vector Machine(SVM) with
 555 both linear and radial basis function (RBF) as the kernel function; 3) Decision
 Tree and Random Forest are tree-based model learning from separating and
 pruning; 4) Adaptive Boosting with the iterative algorithms, and 5) Gaussian
 Naive Bayes. As shown, our method, named Neural Network in the figure,
 achieves the highest classification accuracy among all.

560 7.2. Time Prediction

Here, we compare the job execution time prediction accuracy of our ap-
 proach, *NN*, to three other methods, *EQ*, *Tree-based*, and *SVM*.

As introduced in Section 6.2, *EQ* represents the class of traditional ap-
 proaches that try to construct explicit equations for capturing the effects of
 565 certain key factors, such as the number of cores, tasks, slots, and input data
 size. The equation coefficients are determined by a linear regression method un-
 der a given Hadoop configuration. However, there could be tens or hundreds of
 parameters in the Hadoop configuration. Hence, its prediction accuracy starts
 to deviate when the jobs are running with varied Hadoop configurations. To
 570 illustrate this limitation, Figure 7 compares the prediction error rate when jobs
 are only generated with the default Hadoop configuration versus the results
 when jobs are generated with a randomly selected Hadoop configuration. As
 shown from the results, for jobs with default Hadoop configuration, the error
 rate is less than 25% for the 3 types of jobs we tested. But for jobs with ran-
 575 domly selected Hadoop configuration, the error rate grows another 5%~10%.
 This result indicates the fact that this class of prediction method is unsuitable to
 address our prediction problem with large numbers of features with the complex
 inter-dependency relationship.

Next, we compare the gray-box prediction techniques, including our deep
 580 learning method (*NN*), and previous ML methods (*Tree-based* and *SVM*). Fig-
 ure 8 shows the prediction error rates for each of the 10 types of jobs in our
 experiments. We find that *Tree-based* is more likely to suffer from the over-
 fitting problem because it has a simpler model compared to the others. Take

Kmeans1 as an example; we observe that *Tree-based* has a training error rate of less than 8%, but its testing error rate is up to 23%. As a result, *Tree-based* has the highest error rates for all the job types, except PageRank1 (i.e., the iterated value propagation in PageRank algorithm). This is likely because the data input size of each iteration could change, but our profiler only records the initial input size in the job profile. The results between *SVM* and *NN* are close. However, clearly, *NN* still performs better than *SVM* in most cases, especially for *PageRank2* and *WordCount*. Overall, our method (*NN*) achieves the lowest average prediction error rate among all prediction methods at only 10.9%.

7.3. Individual Job Optimization

Our job optimization method (shown in Algo 1) is designed to find the best resource and Hadoop configurations under a given budget of time or cost. Here, we use *Terasort* as an example to evaluate our optimization method.

First, we show how much job execution time can be reduced by the optimized Hadoop configuration in Figure 9. We plot the job execution time of 9 different problem settings from the combination of 3 different input data sizes (6GB, 12GB, and 18GB) and 3 different resource configurations (3 Small, 3 Medium, and 3 Large instances). The y-axis is the job execution time of optimized configuration normalized to the time of default configuration. Lower values in the plot mean greater time improvement from our optimization. As observed, our optimization significantly reduces job execution time across all test cases. For *Medium* and *Large* VM types, the improvements are even higher than a factor of 2. *Small* VM type has fewer cores and less memory size, thus it limits the parallelism of reducers and the room for optimizations, especially for larger data size. However, we can still observe that the execution time is reduced by 25% and 10% in the case of 12GB and 18GB, respectively. Therefore, optimizing Hadoop configuration is critical to job performance.

Next, we allow our optimizer to decide both the resource configuration and Hadoop configuration under a given cost or time budget. Figure 10a plots the optimized cost under the varied time budget (deadline) constraint, while Fig-

ure 10b plots the optimized job execution time under the varied cost budget
 615 constraint. In these plots, *Large*, *Medium*, *Small* are the results of using only
 one particular VM instance type with the default Hadoop configuration setting.
 Hence, they can only adjust their number of VM instances, and the maximum
 number of instances is limited to 12. In contrast, *Optimized* is the result of our
 approach which can choose different VM instance types and Hadoop configu-
 620 rations under each constraint setting. If a job cannot complete its execution
 within the given time or cost constraint, its result is not plotted in the figures.

For all the methods, the shorter execution time can be reached with a higher
 cost budget, and a lower cost can be reached with a longer deadline. However,
 as observed from both plots, our optimization can achieve significantly better
 625 results under a fixed time or cost constraint. For instance, in order to com-
 plete the job within a deadline of 17 minutes, the running costs of using Large,
 Medium, and Small instances with default Hadoop configuration requires \$1.1,
 \$0.5, and \$0.3, respectively; but in comparison, our optimized result only needs
 \$0.2. Similarly, when a budget of more than \$8,000 is given, naive methods with
 630 default Hadoop configuration cannot finish the job within 700 seconds, but our
 method needs no more than 100 seconds to complete. This set of experiments
 shows the necessity and advantage of optimizing both resource and Hadoop
 configurations.

Our method achieves much better results in Figure 10a and Figure 10b not
 635 just because of choosing better Hadoop configurations, but also choosing better
 resource configurations as well. As shown in Figure 10a, although using Small
 instances can achieve lower costs than using Medium and Large instances with
 a deadline larger than 14 minutes, it cannot finish the job at all if the deadline is
 13 minutes. Therefore, larger instances must be chosen under stricter deadlines.
 640 Our optimized did choose larger instances when it is necessary and optimized
 the Hadoop configuration. That is why it can still complete the job within 3
 minutes with less than \$0.35. Similarly, in Figure 10b, the Small instance has
 a shorter execution time than the Medium between cost \$3,500 $\sim$ \$5,500, but
 the Medium instance has a shorter execution time than the Medium between

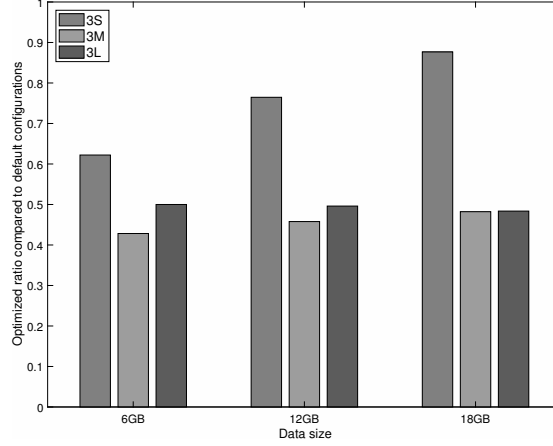
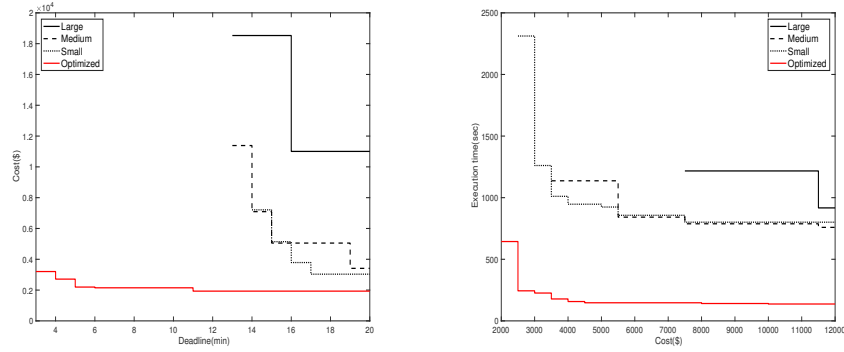


Figure 9: Job time reduction from optimizing Hadoop configurations. *Large, Medium, Small* are the results of using only one particular size of VM instances. *Optimized* is the result from our approach which can choose different VM instance types according to the constraint.



(a) Computing cost comparison under dead- (b) Job time comparison under cost con-
line constraints. straints.

Figure 10: Comparison under the time of cost constraint. *Large, Medium, Small* are the results of using only one particular size of VM instances. *Optimized* is the result from our approach which can choose different VM instance types according to the constraint.

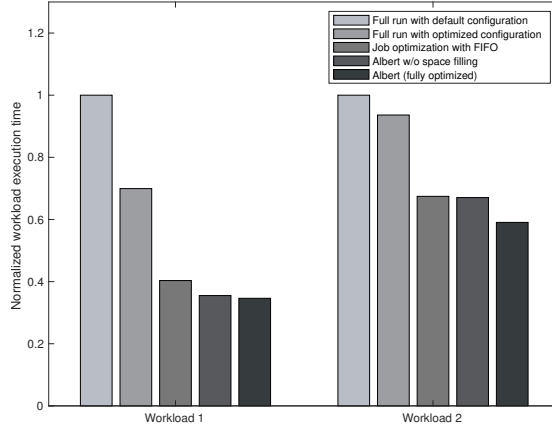


Figure 11: The workload execution time comparison.

cost \$11,500 ~ \$12,000. Also, larger instances may be too expensive to be used under a stricter cost budget. All these results indicate choosing instance type is not a trivial decision for users, but our approach can solve this issue with predicted job performance.

7.4. Overall System Optimization

For the system-level optimization, we evaluate the overall system performance (total execution time) for running two workloads, named by workload 1 and workload 2. Each workload contains 100 jobs. Workload 1 is a more complicated workload consisting of 8 different applications listed in Table 4 and each application can have 4 different input file sizes. On the contrary, workload 2 is a simpler workload that only contains the *WordCount* application with 4 different input file sizes. As a result, the diversity of the job execution time is much larger in workload 1 than workload 2.

In order to show the performance improvement from each of our optimization techniques, we compare the results of the following five different methods. “**Full run with default configuration**” is the result of a naive execution method that simply uses all the resources to run a job with the default Hadoop job configuration setting. It also represents a commonly-seen default execution method when users have no knowledge of the job and system. “**Full run with**

“**optimized configuration**” is an improved method that uses our auto-tuned
665 job configuration strategy for optimizing the execution time of individual jobs.
But without system-level optimization, the jobs are still run with full resources
and scheduled in FIFO order. “**Job optimization with FIFO**” refers to the
result from using our auto-tuned strategy to decide both the resource config-
uration and the Hadoop job configuration of jobs. But the scheduling order
670 remained to be FIFO. Finally, “**Albert (w/o space filling)**” and “**Albert
(fully optimized)**” are the results of our proposed methods with both job
and system optimizations. So the job configuration, resource provisioning, and
job scheduling are all optimized according to the predicted job execution time.
The difference between the two methods is that “Albert (w/o space filling)”
675 doesn’t attempt to fill the space with smaller jobs (set $p = 0$ in our scheduling
algorithm), while “Albert (fully optimized)” uses 10% of the jobs to fill the
scheduling space (set $p = 0.1$).

The results are summarized in Figure 11. The y-axis shows the execution
time of each method normalized to the result of the baseline method “Full run
680 with default configuration”. As shown in the plot, with optimized Hadoop con-
figuration, “Full run with optimized configuration” reduces the execution time
of “Full run with default configuration” by 30% for workload 1, and by 7%
for workload 2. By optimizing both resource configuration and Hadoop config-
uration, “Job optimization with FIFO” further reduce the execution time by
685 60% and 33% for workload 1 and workload, respectively. Finally, with our job
scheduling optimization, the execution time can be further reduced by another
5% ~ 10%. Because workload 1 has larger job diversity than workload 2, we
observed greater optimization improvements in workload 1 than workload 2. On
the other hand, our space-filling scheduling technique shows greater improve-
690 ments in workload 2 than workload 1 indicates the technique can provide further
optimization when the job diversity is not enough to fill the wasted scheduling
space. Overall, all our optimization techniques can contribute to the overall
performance improvements, and 40% ~ 65% time reduction was observed from
our experiment.

695 8. Conclusion

This work addresses the resource and execution configuration problem of running Hadoop workload in the cloud environment. Our approach uses deep learning techniques to predict job execution time. Based on the predicted job information, we design a search algorithm to determine the optimal configurations of a job, and a scheduling algorithm to improve the overall system performance. We evaluate our prediction and optimization methods using real data analytic workloads generated from HiBench. Our study results provide strong evidence that the deep learning technique is well suited for predicting the execution time of a job, especially when the job is managed by a complex computing framework, such as MapReduce. The results also show that our proposed optimization method can effectively find the optimal job configuration and scheduling decision to improve system performance. Although our approach is designed and validated by the Hadoop workload, our method can be generalized for workloads from other parallel processing frameworks as well.

710 References

- [1] Big Data - Global Market Trajectory & Analytics, Global Industry Analysts, Inc, 2020.
- [2] Apache, Apache hadoop, [Online]. Available: <http://hadoop.apache.org/>.
- 715 [3] J. Dean, S. Ghemawat, Mapreduce: Simplified data processing on large clusters, Commun. ACM 51 (1) (2008) 107–113. doi:10.1145/1327452.1327492.
URL <http://doi.acm.org/10.1145/1327452.1327492>
- [4] Apache, Apache hbase, [Online]. Available: <http://hbase.apache.org/>.
- 720 [5] Apache, Apache giraph, [Online]. Available: <http://giraph.apache.org/>.

- [6] A. Thusoo, J. S. Sarma, N. Jain, Z. Shao, P. Chakka, S. Anthony, H. Liu, P. Wyckoff, R. Murthy, Hive: A warehousing solution over a map-reduce framework, *Proc. VLDB Endow.* 2 (2) (2009) 1626–1629. doi:10.14778/1687553.1687609.
 725 URL <https://doi.org/10.14778/1687553.1687609>
- [7] Apache, Apache mahout, [Online]. Available: <http://mahout.apache.org/>.
- [8] P. Lama, X. Zhou, Aroma: Automated resource allocation and configura-
 730 tion of mapreduce environment in the cloud, in: *Proceedings of the 9th International Conference on Autonomic Computing, ICAC '12*, ACM, New York, NY, USA, 2012, pp. 63–72. doi:10.1145/2371536.2371547.
 URL <http://doi.acm.org/10.1145/2371536.2371547>
- [9] S. Kadirvel, J. A. B. Fortes, Grey-box approach for performance prediction
 735 in map-reduce based platforms, in: *2012 21st International Conference on Computer Communications and Networks (ICCCN)*, 2012, pp. 1–9. doi:10.1109/ICCCN.2012.6289311.
- [10] C. O. Chen, Y. Q. Zhuo, C. C. Yeh, C. M. Lin, S. W. Liao, Machine
 740 learning-based configuration parameter tuning on hadoop system, in: *2015 IEEE International Congress on Big Data*, 2015, pp. 386–392. doi:10.1109/BigDataCongress.2015.64.
- [11] H. Yang, Z. Luan, W. Li, D. Qian, G. Guan, Statistics-based workload mod-
 745 eling for mapreduce, in: *2012 IEEE 26th International Parallel and Distributed Processing Symposium Workshops PhD Forum*, 2012, pp. 2043–2051. doi:10.1109/IPDPSW.2012.254.
- [12] G. Wang, A. R. Butt, P. Pandey, K. Gupta, A simulation approach to
 evaluating design decisions in mapreduce setups, in: *IEEE International Symposium on Modeling, Analysis Simulation of Computer and Telecommunication Systems*, 2009, pp. 1–11.

- 750 [13] Z. Zhang, L. Cherkasova, B. T. Loo, Autotune: Optimizing execution
concurrency and resource usage in mapreduce workflows, in: Proceedings
of the 10th International Conference on Autonomic Computing, USENIX,
2013, pp. 175–181.
URL [https://www.usenix.org/conference/icac13/](https://www.usenix.org/conference/icac13/technical-sessions/presentation/zhang_zhuoyao)
755 [technical-sessions/presentation/zhang_zhuoyao](https://www.usenix.org/conference/icac13/technical-sessions/presentation/zhang_zhuoyao)
- [14] H. Altwaijry, E. Trulls, J. Hays, P. Fua, S. Belongie, Learning to match
aerial images with deep attentive architectures, in: 2016 IEEE Conference
on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 3539–
3547. doi:10.1109/CVPR.2016.385.
- 760 [15] R. Giri, M. L. Seltzer, J. Droppo, D. Yu, Improving speech recognition
in reverberation using a room-aware deep neural network and multi-task
learning, in: 2015 IEEE International Conference on Acoustics, Speech and
Signal Processing (ICASSP), 2015, pp. 5014–5018. doi:10.1109/ICASSP.
2015.7178925.
- 765 [16] G. Lample, D. S. Chaplot, Playing FPS games with deep reinforcement
learning, CoRR abs/1609.05521. arXiv:1609.05521.
URL <http://arxiv.org/abs/1609.05521>
- [17] M. R. Wyatt, S. Herbein, T. Gamblin, A. Moody, D. H. Ahn, M. Taufer,
PRIONN: Predicting runtime and io using neural networks, in: Proceedings
770 of the 47th International Conference on Parallel Processing, 2018, pp. 1–12.
- [18] G. Wu, J. L. Greathouse, A. Lyashevsky, N. Jayasena, D. Chiou, GPGPU
performance and power estimation using machine learning, in: IEEE 21st
International Symposium on High Performance Computer Architecture,
2015, pp. 564–576.
- 775 [19] OpenStack, Openstack, [Online]. Available: <http://www.openstack.org/>.

- [20] H. Herodotou, Y. Chen, J. Lu, A survey on automatic parameter tuning for big data processing systems, *ACM Comput. Surv.* 53 (2).
URL <https://doi.org/10.1145/3381027>
- 780 [21] HadoopTuning, [Online]. Available: <http://hadooptutorial.info/hadoop-performance-tuning/> (2015).
- [22] A. Verma, L. Cherkasova, R. H. Campbell, ARIA: Automatic Resource Inference and Allocation for Mapreduce Environments, in: *Proceedings of the 8th ACM International Conference on Autonomic Computing*, 2011, pp. 235–244.
- 785 [23] H. Herodotou, H. Lim, G. Luo, N. Borisov, L. Dong, F. B. Cetin, S. Babu, Starfish: A self-tuning system for big data analytics, in: *In CIDR*, 2011, pp. 261–272.
- [24] A. Gandhi, S. Thota, P. Dube, A. Kochut, L. Zhang, Autoscaling for hadoop clusters, in: *2016 IEEE International Conference on Cloud Engineering (IC2E)*, 2016, pp. 109–118. doi:10.1109/IC2E.2016.11.
- 790 [25] Z. Zhang, L. Cherkasova, B. T. Loo, Benchmarking approach for designing a mapreduce performance model, in: *Proceedings of the 4th ACM/SPEC International Conference on Performance Engineering, ICPE '13*, ACM, 2013, pp. 253–258. doi:10.1145/2479871.2479906.
URL <http://doi.acm.org/10.1145/2479871.2479906>
- 795 [26] Z. Bei, Z. Yu, H. Zhang, W. Xiong, C. Xu, L. Eeckhout, S. Feng, Rfhoc: A random-forest approach to auto-tuning hadoop’s configuration, *IEEE Transactions on Parallel and Distributed Systems* 27 (5) (2016) 1470–1483. doi:10.1109/TPDS.2015.2449299.
- 800 [27] M. Li, L. Zeng, S. Meng, J. Tan, L. Zhang, A. R. Butt, N. Fuller, MRONLINE: MapReduce Online Performance Tuning, in: *Proceedings of the 23rd International Symposium on High-Performance Parallel and Distributed Computing, HPDC '14*, Association for Computing Machinery, New York,

- 805 NY, USA, 2014, p. 165176. doi:10.1145/2600212.2600229.
URL <https://doi.org/10.1145/2600212.2600229>
- [28] H. Herodotou, F. Dong, S. Babu, No one (cluster) size fits all: Automatic cluster sizing for data-intensive analytics, in: Proceedings of the 2nd ACM Symposium on Cloud Computing, SOCC '11, Association for Computing Machinery, New York, NY, USA, 2011. doi:10.1145/2038916.2038934.
810 URL <https://doi.org/10.1145/2038916.2038934>
- [29] N. Yigitbasi, T. L. Willke, G. Liao, D. Epema, Towards machine learning-based auto-tuning of mapreduce, in: 2013 IEEE 21st International Symposium on Modelling, Analysis and Simulation of Computer and Telecommunication Systems, 2013, pp. 11–20. doi:10.1109/MASCOTS.2013.9.
815
- [30] K. Singh, E. undefinedpek, S. A. McKee, B. R. de Supinski, M. Schulz, R. Caruana, Predicting parallel application performance via machine learning approaches: Research articles, *Concurr. Comput.: Pract. Exper.* 19 (17) (2007) 22192235.
- 820 [31] T.-P. Pham, J. J. Durillo, T. Fahringer, Predicting workflow task execution time in the cloud using a two-stage machine learning approach, *IEEE Transactions on Cloud Computing* 8 (1) (2020) 256–268. doi:10.1109/TCC.2017.2732344.
- [32] J. Bi, H. Yuan, M. Zhou, Q. Liu, Time-dependent cloud workload forecasting via multi-task learning, *IEEE Robotics and Automation Letters* 4 (3)
825 (2019) 2401–2406. doi:10.1109/LRA.2019.2899224.
- [33] D. Nemirovsky, T. Arkose, N. Markovic, M. Nemirovsky, O. Unsal, A. Cristal, A machine learning approach for performance prediction and scheduling on heterogeneous cpus, in: 2017 29th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD), 2017, pp. 121–128. doi:10.1109/SBAC-PAD.2017.23.
830
- [34] EMR, EMR, [Online]. Available: <https://aws.amazon.com/tw/emr/>.

- [35] AWS, Aws, [Online]. Available: <https://aws.amazon.com/tw/>.
- [36] Shara, Shara project, [Online]. Available: <https://wiki.openstack.org/wiki/Sahara/>.
835
- [37] S. Babu, H. Herodotou, Massively parallel databases and mapreduce systems, *Found. Trends Databases* 5 (1) (2013) 1104. doi:10.1561/19000000036.
URL <https://doi.org/10.1561/19000000036>
- 840 [38] Rackspace, Rackspace: Managed dedicated & cloud computing services, [Online]. Available: <https://www.rackspace.com/>.
- [39] V. K. Vavilapalli, A. C. Murthy, C. Douglas, S. Agarwal, M. Konar, R. Evans, T. Graves, J. Lowe, H. Shah, S. Seth, B. Saha, C. Curino, O. O'Malley, S. Radia, B. Reed, E. Baldeschwieler, Apache Hadoop YARN: Yet Another Resource Negotiator, in: *Proceedings of the 4th Annual Symposium on Cloud Computing*, 2013, pp. 5:1–5:16.
845
- [40] A. Lodi, S. Martello, D. Vigo, Recent advances on two-dimensional bin packing problems, *Discrete Applied Mathematics* 123 (1) (2002) 379–396. doi:[https://doi.org/10.1016/S0166-218X\(01\)00347-X](https://doi.org/10.1016/S0166-218X(01)00347-X).
850 URL <https://www.sciencedirect.com/science/article/pii/S0166218X0100347X>
- [41] D. Pisinger, M. Sigurd, The two-dimensional bin packing problem with variable bin sizes and costs, *Discrete Optimization* 2 (2) (2005) 154–167. doi:<https://doi.org/10.1016/j.disopt.2005.01.002>.
855 URL <https://www.sciencedirect.com/science/article/pii/S1572528605000216>
- [42] J. Jynlki, A thousand ways to pack the bin – a practical approach to two-dimensional rectangle bin packing (2010).
- [43] S. Huang, J. Huang, J. Dai, T. Xie, B. Huang, The HiBench benchmark suite: Characterization of the MapReduce-based data analysis, in:
860

2010 IEEE 26th International Conference on Data Engineering Workshops
(ICDEW 2010), 2010, pp. 41–51. doi:10.1109/ICDEW.2010.5452747.